

HuskyMingle: A Cross-Platform, Native-Client Architecture for a Verified-Identity Campus Social Network

Kaustubha Venkata Eluri
Team Lead
Northeastern University
Boston, MA, USA

Abstract—Northeastern University students rely on a fragmented set of applications to manage campus life: a photo-sharing app for social posts, a chat platform for club communities, a peer-to-peer marketplace for textbooks, a jobs platform for internships, a matching app for making friends, and separate tools for events and study groups. None of these platforms verify that a user is an enrolled student, and none share data with one another. This paper documents HuskyMingle, a campus social network built from scratch across three independently deployable clients — a Next.js/NestJS web application, a native Swift/SwiftUI iOS application, and a native Kotlin/Jetpack Compose Android application — all sharing a single stateless NestJS backend, a PostgreSQL database (via Prisma), and a Redis cache/pub-sub layer. We describe the shared-backend, native-multi-client architecture; the feature surface (twenty-plus authenticated routes including a social feed, ephemeral Stories, real-time messaging, peer matching, communities, private Circles, a course catalog, marketplace, and gamification); and the engineering decisions that diverge across platforms, drawn directly from each sub-project’s architecture decision records (ADRs). We report only outcomes that are documented in the project’s own artifacts, and we present an honest limitations analysis — drawn from each platform’s `limitations.md` — rather than a promotional account. The result is a case study in building one product surface across three client stacks while keeping a single backend contract, useful to teams facing the same web/iOS/Android fan-out problem.

Index Terms—cross-platform architecture, native mobile development, campus social network, client-server design, mobile state management, software architecture decision records

I. INTRODUCTION

Campus social life is mediated today by a patchwork of general-purpose consumer applications repurposed for university use. A Northeastern University student who wants to post an update, join a club, sell a used textbook, apply to an internship, find a study partner, and RSVP to an event today needs at least six separate applications — typically Instagram, Discord, Facebook Marketplace, Handshake, Bumble BFF, and a mix of ad-hoc event tools. Each of these applications was designed for a general population, not a university population; none verifies that the person on the other side of a conversation is an actual, enrolled student; and none of them share a single identity or a single social graph with the others. This fragmentation increases onboarding friction, dilutes trust (a “student marketplace” posting on general Facebook Marketplace is not

actually restricted to students), and duplicates social graphs that could otherwise be unified around a single verified .edu identity.

HuskyMingle is a full-stack campus social network built to close this gap: a single, brand-native product that consolidates the feed, peer matching, threaded discussion, marketplace, ephemeral Stories, a real course catalog, and private group (“Circles”) functionality behind one verified university identity, and that is available natively on the three surfaces students actually use — the web, iOS, and Android. Rather than building one cross-platform codebase (e.g., a single Flutter or React Native application) and accepting its platform-fidelity compromises, the project deliberately built three native or near-native clients (a Next.js web client, a zero-dependency native SwiftUI iOS client, and a zero-Hilt native Jetpack Compose Android client) against one shared NestJS backend. This paper documents that architecture, the platform-specific engineering tradeoffs that were made and recorded as ADRs on each client, and an honest accounting of what is and is not finished, based entirely on the project’s own README and documentation files.

The contribution of this paper is not a novel algorithm but a documented case study: how a single backend contract can be served to three independently engineered native clients, what state-management and security patterns each platform converges on or diverges on, and what the real (not aspirational) surface-by-surface completion state looks like across a three-client social product.

II. RELATED WORK

A. General Social Networking Platforms

General-purpose social networks (photo/video sharing platforms, chat-community platforms, and marketplace apps) provide the individual feature primitives that HuskyMingle consolidates — a chronological or ranked feed, ephemeral photo “stories,” direct and group messaging, and peer-to-peer commerce listings. These platforms are horizontally scaled, identity-agnostic (any email or phone number can register), and monetized through advertising or marketplace take-rates. Campus-specific networks differ in a key structural way: the target population is closed and independently verifiable (a

university-issued .edu address), which allows a smaller, trust-dense social graph and removes the need for the anti-spam and anti-bot infrastructure that open networks require. HuskyMingle’s onboarding flow — .edu email, one-time six-digit verification code, then interest/skill/language personalization — is a direct consequence of this closed-population design choice.

B. Cross-Platform Architecture Patterns

Teams that must ship a product on web, iOS, and Android broadly choose between three architectural strategies: (1) a single cross-platform UI toolkit (e.g., Flutter or React Native) compiled or interpreted across all target platforms; (2) native clients per platform sharing a common backend API contract; or (3) a hybrid where a native shell wraps a web view. Cross-platform toolkits reduce duplicated UI logic at the cost of platform fidelity and first-paint latency, particularly on older devices, and often lag behind a platform’s own UI idioms. HuskyMingle’s iOS sub-project documents exactly this tradeoff: the wider HuskyMingle effort already contains a Flutter mobile client, but the iOS ADR record (ADR-001) reports that the Flutter client’s first-paint p95 exceeded 1.2 seconds on older devices and its Cupertino-styled UI did not track iOS 17 idioms such as `ContentUnavailableView` or native `.refreshable`. The project’s response was strategy (2): independent native clients (SwiftUI on iOS, Jetpack Compose on Android) that consume the identical NestJS REST/WebSocket contract also served to the Next.js web client, so no client requires a bespoke backend change. This mirrors an industry-common pattern of “API-first, native-everywhere” architecture, in which the backend is the single point of contract stability and each client is free to choose its own idiomatic state-management and persistence strategy.

III. SYSTEM ARCHITECTURE

A. Overview

HuskyMingle is organized as three independent repositories that share one backend contract:

- **HuskyMingle-web** — a Next.js 15 (App Router) frontend and a NestJS 10 API in a single repository, deployed together via Docker Compose.
- **HuskyMingle-iOS** — a native Swift/SwiftUI client with zero third-party dependencies, targeting iOS 17+.
- **HuskyMingle-Android** — a native Kotlin/Jetpack Compose client with a narrow, curated dependency footprint, targeting Android 8.0+ (API 26), compiled against API 35.

All three clients speak to the same stateless NestJS backend over REST, and the web client additionally maintains a persistent Socket.io connection for real-time feed and messaging events. The backend is fronted by Redis for pub/sub and session/cache duties and persists to PostgreSQL 16 through Prisma, whose schema spans 40+ models and 12 enums across 24 backend feature modules exposing 80+ REST endpoints.

Fig. 4 shows the social feed screen as independently rendered by each of the three native clients, illustrating the shared

feature surface reached through platform-idiomatic UI: the Next.js web client (Fig. 1), the SwiftUI iOS client (Fig. 2), and the Jetpack Compose Android client (Fig. 3).

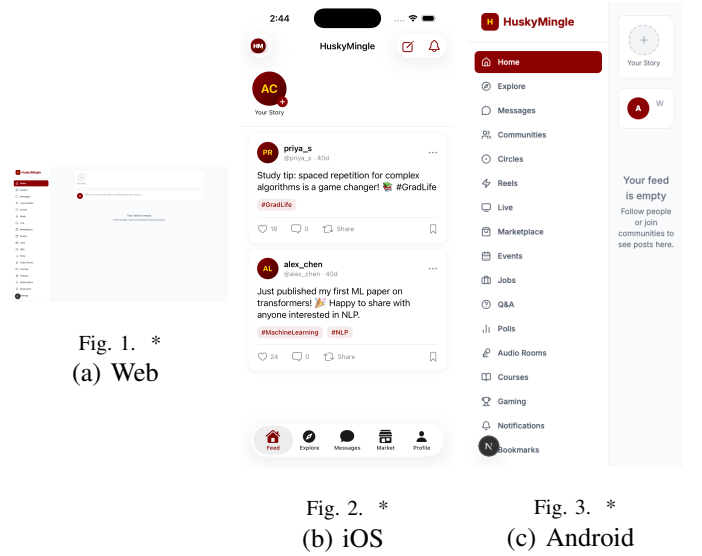


Fig. 4. The social feed screen rendered natively across all three HuskyMingle clients: (a) web, (b) iOS, (c) Android.

B. Backend

The NestJS backend is the single source of truth for all business logic and the only component that touches the database. Authentication uses Passport.js-issued JWTs: a short-lived access token (15-minute TTL, configurable via `JWT_EXPIRES_IN`) paired with a separate refresh-token secret and rotation flow. A per-message AI translation pipeline for real-time chat falls back gracefully from GPT-4o-mini, to Google Translate, to a local mock, if upstream credentials or services are unavailable. The Prisma seed script populates roughly 20 demonstration users, posts, communities, and fifty real Northeastern University course records, which all three clients also bundle or query for their course-catalog surface.

The web repository’s own tradeoff notes explain why the API layer is a separate NestJS service rather than colocated Next.js API routes: Next.js route handlers are reachable only from the Next.js server process, which would couple the backend to the web framework and block the two native mobile clients from ever sharing it. Decoupling the API into its own NestJS process means the iOS and Android clients require no backend changes to consume the same contract.

C. Web Client

The Next.js 15 web client uses the App Router with a route-grouped structure separating unauthenticated flows (`((auth))`: login, register, verify, onboarding) from an authenticated shell (`((main))`) containing twenty feature routes reachable from a fixed left sidebar. Server state (feed pagination, background refetch) is owned by TanStack (React) Query; a thin layer of genuinely client-only state (auth session, private

Circles data) is owned by Zustand with persist middleware to `localStorage`. An Axios instance centralizes JWT attachment and silently retries a request once after a 401-triggered token refresh. A lazily instantiated Socket.io singleton (`autoConnect: false`) authenticates over an `authenticate` event once a JWT is available and drives real-time new-post, typing, and read-receipt events.

D. iOS Client

The iOS client follows a strict, unidirectional View → ViewModel → Service → Model layering. Views own no networking or Codable logic; `@MainActor ObservableObject` ViewModels are used selectively for the screens with genuine state-machine complexity (Feed, Explore, Chat, User Profile, Post Detail), while long-tail screens (Polls, Q&A, Bookmarks, etc.) use inline `@State` for one-line load logic. A single `APIService` is the only class in the codebase that touches `URLSession`; a single `onUnauthorized` callback, wired once in `AuthService.init`, is the only path by which a 401 response can log the user out. The client ships with *zero* third-party dependencies — no SPM packages, no CocoaPods — relying entirely on Apple frameworks and the Swift standard library. Authentication tokens are stored in the iOS Keychain under `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly` with a one-time, backward-compatible migration path that lifts any legacy `UserDefaults`-stored token into the Keychain without forcing a re-login.

E. Android Client

The Android client mirrors the iOS client’s layered separation almost one-for-one: Composable screens are presentation-only, ViewModels expose `StateFlow`, and a single `APIService` Retrofit interface (behind `RetrofitClient`, with an `AuthInterceptor` and Gson serialization) is the only place HTTP requests are constructed. Rather than a dependency-injection framework such as Hilt, the app uses a manual Application-level singleton container (`HuskyMingleApp.kt`) exposing `lateinit` references to its data stores; the project’s own documentation states this is deliberate to keep cold-start cost low, with an explicit revisit trigger (“if the app grows past ~50 ViewModels”). Authentication tokens and user-scoped preferences live in `AndroidX DataStore<Preferences>` rather than `SharedPreferences`. Biometric app-lock uses `AndroidX BiometricPrompt` restricted to `BIOMETRIC_STRONG | DEVICE_CREDENTIAL` authenticators only, with no weak-biometric fallback.

F. Shared Cross-Platform Design Language

Both native clients converge on an identical Northeastern “Husky” brand token set — a husky-red primary (`#8B0000`), a deep-red gradient anchor (`#4A0000`), a gold accent (`#FFD700`) — expressed through platform-idiomatic single-source-of-truth files (`AppTheme.swift` on iOS; the

`ui/theme/` package on Android), an identical 4-point spacing scale (4/8/12/16/20/24/32/48), and an identical corner-radius scale (8/12/16/20/pill). Both clients also converge on the same defensive-decoding pattern: because the NestJS backend was built incrementally, different list endpoints return payloads shaped as `{ posts: [...] }, { items: [...] }, { data: [...] }`, or a bare JSON array. Rather than force a coordinated three-client backend schema change, each native client implements a client-side “resolve whichever key is present” wrapper (a `ListResponse` protocol on iOS; an equivalent list-always Retrofit contract on Android), collapsing what would otherwise be a duplicated fallback block in every list screen into a single reusable helper.

IV. KEY CROSS-PLATFORM ENGINEERING DECISIONS

This section summarizes the architecture decision records (ADRs) and documented tradeoffs recorded per platform.

A. Native Clients over a Single Cross-Platform Toolkit

As discussed in Section II-B, the iOS ADR record (ADR-001, dated 2026-05) documents the explicit choice of native SwiftUI over extending the project’s pre-existing Flutter mobile client, citing a first-paint p95 of ≥ 1.2 s on older devices and non-native Cupertino styling as the deciding factors. The acknowledged cost is now maintaining two (and, with Android, three) separate mobile clients; this is mitigated because none of them require backend changes, so the marginal engineering cost of a new client is confined to that client’s own repository.

B. State Management Divergence by Platform Idiom

Each client chose the state-management pattern idiomatic to its platform rather than importing a shared abstraction:

- **Web:** TanStack Query for server state (cache, background refetch, pagination) plus Zustand for the residual client-only state (auth session, private Circles), explicitly justified in the web README as “Zustand only touches what React Query can’t.”
- **iOS:** `@MainActor ObservableObject` ViewModels with `@Published` properties, applied selectively (ADR-004, “Light MVVM”) rather than uniformly across all ~30 views, because full MVVM on every long-tail screen was assessed as $2\times$ the implementation cost for no measurable benefit on screens with one-line load logic.
- **Android:** `androidx.lifecycle.ViewModel` plus Kotlin `StateFlow`, described in the Android README as “the same MVVM contract as iOS,” with the identical selective-ViewModel tradeoff documented for its own long-tail screens.

C. Auth Token Storage per Platform

Each client picked the storage primitive appropriate to its own security model rather than a shared cross-platform library:

- **Web:** JWTs are persisted in `localStorage` through the Zustand persist middleware; an Axios response interceptor detects a 401, silently exchanges the refresh token, and retries the original request once.

- **iOS:** JWTs are persisted in the Keychain under `kSecAttrAccessibleAfterFirstUnlockThisDevice` rather than special-casing each screen — documented on iOS as ADR-005 and mirrored, per the Android README, as an equivalent contract at the Retrofit-interface layer. The iOS documentation reports this collapsed fourteen duplicate try/catch fallback blocks into a single `getList` helper.
- **Android:** JWTs are persisted in `AndroidX DataStore<Preferences>`, which is explicitly flagged in the Android README’s own tradeoffs section as *not* encrypted at rest (unlike the iOS Keychain choice) — the documented plan, prior to any real release, is to wrap the token in `EncryptedSharedPreferences` or a Keystore-backed AEAD key. This is a rare case where the two native clients diverge in security posture, and it is called out candidly in the project’s own documentation rather than glossed over.

D. Dependency Minimalism as a Deliberate Constraint

Both native clients treat dependency count as a design variable rather than a default. iOS ships with literally zero third-party packages (ADR-002), and Android’s dependency footprint is described as “intentionally narrow” (Compose BOM, Retrofit/OkHttp/Gson, DataStore, Coil, AndroidX Biometric) with an explicit list of frameworks deliberately excluded: no Hilt, no Room, no RxJava, no Glide. Both documents record the trigger condition under which this constraint would be revisited (image-upload multipart handling or WebSocket realtime on iOS; crossing roughly 50 ViewModels or 5 feature modules on Android).

E. Client-Side “Ship Now, Replace Later” Features

Three feature surfaces — Stories, Circles, and the NEU course catalog — are implemented as fully functional, purely client-side features on both mobile clients, ahead of backend persistence support. On iOS, Stories are JPEGs written to `Documents/stories/` with metadata in `UserDefaults` and an automatic 24-hour expiry sweep on launch; Circles are JSON-encoded groups in `UserDefaults`; the course catalog is a bundled `neu_courses.json` of fifty real Northeastern courses. Android mirrors this pattern almost exactly, substituting `DataStore`-backed JSON blobs and its own bundled `assets/neu_courses.json`. Both platforms document this explicitly as a deliberate seam: the view/Composable layer will not change when backend persistence lands, only the internals of the `StoriesStore / CirclesStore / CourseCatalogService` classes will.

F. Defensive API Response Handling

Because the shared NestJS backend evolved incrementally across three consumer clients, individual endpoints return inconsistent list-payload shapes. Both native clients independently converged on the same mitigation strategy — resolve

a set of candidate keys in a single reusable decoding helper rather than special-casing each screen — documented on iOS as ADR-005 and mirrored, per the Android README, as an equivalent contract at the Retrofit-interface layer. The iOS documentation reports this collapsed fourteen duplicate try/catch fallback blocks into a single `getList` helper.

V. EVALUATION

Because this project is a documented engineering case study rather than a deployed, measured production system, we report only outcomes that are directly stated in the project’s own artifacts, and we do not claim user-count, retention, or performance metrics that are not written down in the repositories.

A. Quality Gates

Each client repository documents an explicit, met set of quality gates rather than an automated test suite with numeric coverage. The iOS client’s stated gates include: a clean `xcodebuild` against the iOS 17 simulator SDK; zero raw `Color(red:green:blue:)` literals outside the single design-system file; zero `UserDefaults`-based token storage; App Transport Security scoped to `localhost` only (no blanket `NSAllowsArbitraryLoads`); every asynchronous path using `async/await` with no Combine or completion-handler bridging; and a single shared `AuthService` instance app-wide. The Android client’s stated gates include a clean `./gradlew assembleDebug` and `./gradlew lint` run with zero errors; zero raw `Color(0xFFxxxxxx)` literals outside the theme package; exclusive use of `Icons.AutoMirrored.*` for RTL-safe iconography; an `OkHttp AuthInterceptor` reading a `@Volatile` cached token (avoiding a blocking `runBlocking{}` call on the network thread and its associated ANR risk); and `BiometricPrompt` restricted to strong-or-device-credential authenticators only. Both CI pipelines are documented via GitHub Actions badges in each README.

B. Feature Completeness

The top-level project README documents a feature parity matrix across all three clients: authentication (JWT + OTP), onboarding, the social feed with reactions, 24-hour Stories, real-time messaging, explore/matching, communities, private Circles, events, jobs/internships, marketplace, a course catalog, polls, Q&A, audio rooms, reels, live streams, a gamification system, notifications, bookmarks, search, and dark mode are reported as present (✓) on all three clients; biometric lock is reported as present on iOS and Android only (not applicable to the web client). The web repository additionally documents concrete scale figures for its own implementation: 20 authenticated routes, 24 backend modules, 40+ Prisma models, 80+ API endpoints, 60+ TypeScript source files, and approximately 8,500 lines of code.

C. What Actually Works Today

Rather than presenting the feature matrix uncritically, both mobile READMEs include a “What Works Today” table distinguishing *Live* functionality from *client-side demo-grade* functionality and from *list-only MVP* functionality. Authentication, session restore, the feed, Explore/matching, Followers/Following lists, and Marketplace are reported as fully live end-to-end against the backend on both mobile clients. Gaming, Audio Rooms, Reels, and Live Streams are explicitly reported as list-only on both platforms (no join flow, no WebRTC session, no video player). This self-reported honesty is itself a notable engineering-practice outcome of the project: every feature claim in the top-level feature matrix is qualified, per platform, by a corresponding “what’s actually implemented” statement in that platform’s own documentation.

D. External Recognition

While the project repositories themselves do not record award information, the author’s institutional record documents that Husky Mingle received a Northeastern University hackathon recognition in the Student Networking Track (2024). This recognition speaks to the project’s reception as an early-stage MVP rather than to any of the production-readiness or quantitative claims evaluated above, which remain scoped strictly to what the repositories document.

VI. LIMITATIONS

We report limitations exactly as documented per platform, without embellishment.

A. Cross-Platform

No client implements automated end-to-end or unit test suites at the time of writing (the web roadmap lists Playwright and Jest/RTL as pending; the iOS roadmap lists a `URLProtocol`-mocked test target as its next deliverable). No client has analytics or crash-reporting instrumentation (no Mixpanel, Amplitude, or Sentry). No client implements localization; all user-facing strings are inline English.

B. Web

Image upload is not wired to any object storage (no pre-signed S3 flow yet); push notifications (Web Push API) are not implemented; a deployment configuration for Vercel/Railway is not yet in place; infinite-scroll pagination is backend-wired (cursor-based) but not yet consumed on the frontend.

C. iOS

There is no realtime transport on the client: although the backend exposes a `Socket.io` gateway, the iOS client does not subscribe to it, so chat is pull-on-appear only, with no typing indicators or live message arrival. There are no push notifications (no APNs certificate or `UNUserNotificationCenter` registration). There is no offline cache; every backend-backed screen fetches fresh on each appearance, so a cold launch without network shows an error state on every backend-dependent tab (Stories, Circles,

and Courses still render because they are device-local). List endpoints are capped at 20 items with no pagination. Threaded comments support only one level of nesting via the API, though the UI is built for arbitrary depth. Stories, Circles, and course enrollment state are device-local and do not sync across a user’s installs.

D. Android

The Android limitations mirror iOS closely, with two additional, Android-specific items documented candidly in its own README: pagination is “half-wired” (the feed response model already carries a `nextCursor`, but the `LazyColumn` does not yet consume it), and the bundled `gradlew` wrapper script is documented as broken on modern macOS `zsh` shells, requiring either direct invocation of the Gradle wrapper JAR or regeneration of the script. Tokens are stored in plain (non-encrypted) `DataStore` rather than `EncryptedSharedPreferences` or a `Keystore`-backed key, which the README explicitly flags as unacceptable for a real release build and earmarks for remediation before shipping. Image attachments on posts are not wired (the `Post.images` model field exists, but the composer is text-only).

VII. CONCLUSION AND FUTURE WORK

HuskyMingle demonstrates that a single-team engineering effort can deliver one coherent product surface across three natively engineered clients — web, iOS, and Android — by anchoring all three to one stable, versioned NestJS backend contract, rather than by adopting a single cross-platform UI toolkit. The project’s own documentation shows each platform making platform-idiomatic choices (Zustand/React Query on web, selective MVVM with Keychain-backed tokens on iOS, `StateFlow` with `DataStore`-backed tokens and no dependency-injection framework on Android) while converging independently on the same defensive API-decoding pattern and the same “ship client-side now, wire to backend later” strategy for three demo-grade feature surfaces (Stories, Circles, and the course catalog). The project’s honesty about what is and is not finished — documented per-surface, per-platform, in each repository’s own `limitations.md` — is itself a useful artifact for any team scoping a similar three-client build.

The documented roadmaps converge on a consistent near-term priority ordering across platforms: backend persistence for Stories and Circles (the “swap point” both mobile clients were deliberately built around), `WebSocket`-backed real-time chat to replace pull-on-appear polling, an automated test target, and hardened production authentication (Sign in with Apple on iOS; encrypted token storage and Sign in with Google on Android). Longer-term, both native clients note offline caching (`SwiftData` on iOS; a `Room` layer on Android), push notification wiring (APNs and FCM respectively), and analytics instrumentation as unaddressed. Future work on the shared backend itself would include normalizing the inconsistent list-response shapes that currently require client-side defensive decoding on every native client, ideally via `OpenAPI`-

driven client code generation, which the iOS documentation identifies as the natural point at which today's per-model `ListResponse` wrappers could be retired.